

COMP5339

Data Engineering

Semester 1 - 2026

Group Project 2

Group Members

Group number	Student ID	Name
Tut10 Group96	540430668	Zongquan Tang
Tut10 Group96	500097430	Wei Gong
Tut10 Group96	540768044	Raina Hong

1 System Overview

This project developed a real-time electricity generation and emissions streaming system based on OpenElectricity data for Australian NEM facilities. The system integrated data acquisition, cleaning, MQTT streaming, continuous replay execution, and interactive dashboard visualisation into a single workflow.

The pipeline first retrieved facility metadata, power generation, and CO2 emissions data from the OpenElectricity API. After cleaning and integration, the processed dataset was converted into MQTT JSON messages and continuously replayed to simulate a live streaming environment. A Streamlit dashboard subscribed to the MQTT topic and visualised incoming facility-level updates dynamically on an interactive map.

The final system integrated OpenElectricity API retrieval, MQTT publishing and subscription, continuous execution replay, DuckDB integration, Streamlit dashboard visualisation, and geospatial facility mapping.

2 Data Acquisition, Cleaning and Integration

This project used electricity generation and emissions data from the OpenElectricity platform. We focused on operating facilities in the Australian NEM network and retrieved 5-minute interval data from 9 May 2026 to 16 May 2026.

Facility metadata including facility name, facility code, latitude, and longitude was first retrieved from the OpenElectricity API. The facility code was then used to request both power generation and CO2 emissions time-series data. Since both datasets shared the same facility identifier, interval, and timestamp structure, the two streams could be integrated directly during retrieval rather than merged afterwards manually.

To preserve the original granularity of the electricity system, unit-level information was retained throughout the pipeline. Many facilities contain multiple generating units, so the `unit_code` returned by the API was stored together with the facility identifier and timestamp. As a result, each row in the integrated dataset represented one facility unit at one timestamp with its associated power generation and emissions values.

After acquisition, the raw dataset was cleaned before streaming. Rows containing missing critical fields were removed to prevent incomplete MQTT messages during replay. Duplicate rows were also removed to avoid repeated message publishing. Timestamp fields were converted into datetime format and sorted chronologically to support event-time ordered streaming.

The cleaned dataset was materialised into `integrated_power_emissions_data.csv`. Using a cached CSV avoided repeated API calls during development and allowed the streaming pipeline to be replayed consistently.

Preserving unit-level information throughout the integration and streaming pipeline improved the granularity and accuracy of the final dashboard visualisation.

3 MQTT Streaming and Continuous Execution

The MQTT streaming component simulated a real-time electricity monitoring environment using the cleaned integrated dataset. MQTT publishing was implemented using the `paho-mqtt` Python library, while a local Mosquitto broker running on port 1883 handled communication between the publisher, subscriber, and dashboard.

It is important to emphasize that we found that public MQTT brokers occasionally caused connection failures, particularly under university or public Wi-Fi networks where access to port 1883 could be restricted during development. To improve reliability and ensure reproducible execution, the project adopted a locally hosted Mosquitto broker using:

```
MQTT_BROKER = "localhost"
```

Mosquitto was installed locally and started in a separate terminal before running the publisher and Streamlit dashboard processes. This local deployment approach significantly improved the stability of MQTT message publishing, subscription, and continuous dashboard updates during testing.

Using a locally hosted Mosquitto broker significantly improved connection stability and reproducibility compared with public MQTT brokers under restricted network environments.

Then, messages were published to the topic:

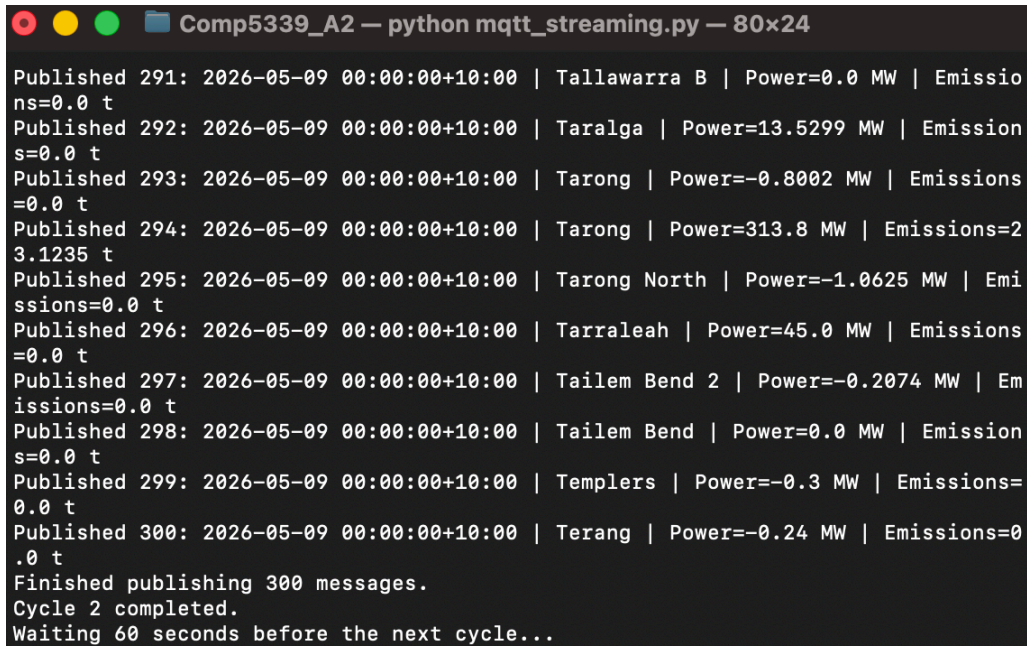
```
comp5339/A2group26/electricity/facility_stream
```

Each MQTT message contained the facility name, facility code, unit code, timestamp, power generation value, emissions value, and facility coordinates. The inclusion of location information supported downstream dashboard visualisation and map rendering.

Before publishing, the dataset was sorted by timestamp, facility code, and unit code to ensure event-time ordered replay. MQTT messages were then streamed one at a time with a 0.1-second delay between messages to simulate live streaming behaviour rather than bulk transmission.

To satisfy the assignment requirement for continuous execution, the publisher was wrapped inside a replay loop. After each streaming cycle completed, the program paused for 60 seconds before restarting the replay process. This simulated an unbounded streaming pipeline while avoiding excessive API requests during development.

Figure 1 shows the MQTT publisher continuously replaying facility-level electricity and emissions records.

A terminal window titled "Comp5339_A2 — python mqtt_streaming.py — 80x24" displays the output of a Python script. The output shows a series of 10 published messages (291 to 300) for various facilities, each containing a timestamp, facility name, power value in MW, and emissions value in tonnes. The messages are published at regular intervals. After message 300, the script reports "Finished publishing 300 messages. Cycle 2 completed. Waiting 60 seconds before the next cycle..."

```
Published 291: 2026-05-09 00:00:00+10:00 | Tallawarra B | Power=0.0 MW | Emissions=0.0 t
Published 292: 2026-05-09 00:00:00+10:00 | Taralga | Power=13.5299 MW | Emissions=0.0 t
Published 293: 2026-05-09 00:00:00+10:00 | Tarong | Power=-0.8002 MW | Emissions=0.0 t
Published 294: 2026-05-09 00:00:00+10:00 | Tarong | Power=313.8 MW | Emissions=23.1235 t
Published 295: 2026-05-09 00:00:00+10:00 | Tarong North | Power=-1.0625 MW | Emissions=0.0 t
Published 296: 2026-05-09 00:00:00+10:00 | Tarraleah | Power=45.0 MW | Emissions=0.0 t
Published 297: 2026-05-09 00:00:00+10:00 | Taillem Bend 2 | Power=-0.2074 MW | Emissions=0.0 t
Published 298: 2026-05-09 00:00:00+10:00 | Taillem Bend | Power=0.0 MW | Emissions=0.0 t
Published 299: 2026-05-09 00:00:00+10:00 | Templers | Power=-0.3 MW | Emissions=0.0 t
Published 300: 2026-05-09 00:00:00+10:00 | Terang | Power=-0.24 MW | Emissions=0.0 t
Finished publishing 300 messages.
Cycle 2 completed.
Waiting 60 seconds before the next cycle...
```

Figure 1: MQTT publisher continuously streaming facility records

The terminal output confirmed that records were published successfully with correct timestamps, facility identifiers, power values, and emissions values. The continuous replay loop also entered the required 60-second waiting state correctly after each cycle.

A subscriber component was additionally implemented to verify that MQTT messages were correctly transmitted and decoded. The subscriber successfully received and displayed real-time facility-level updates from the broker.

The successful real-time replay of facility-level electricity and emissions records confirmed that the MQTT streaming pipeline functioned correctly under continuous execution conditions.

4 Interactive Dashboard Visualisation

A Streamlit-based dashboard was developed to visualise streaming electricity generation and emissions data in real time. The dashboard subscribed directly to the MQTT topic and dynamically updated facility records as new messages arrived.

The dashboard contained two main components: a live facility data table and an interactive map-based visualisation. The interface also provided a metric selector that allowed users to switch between power output mode and CO2 emissions mode.

Figure 2 shows the main dashboard interface after successful MQTT integration.

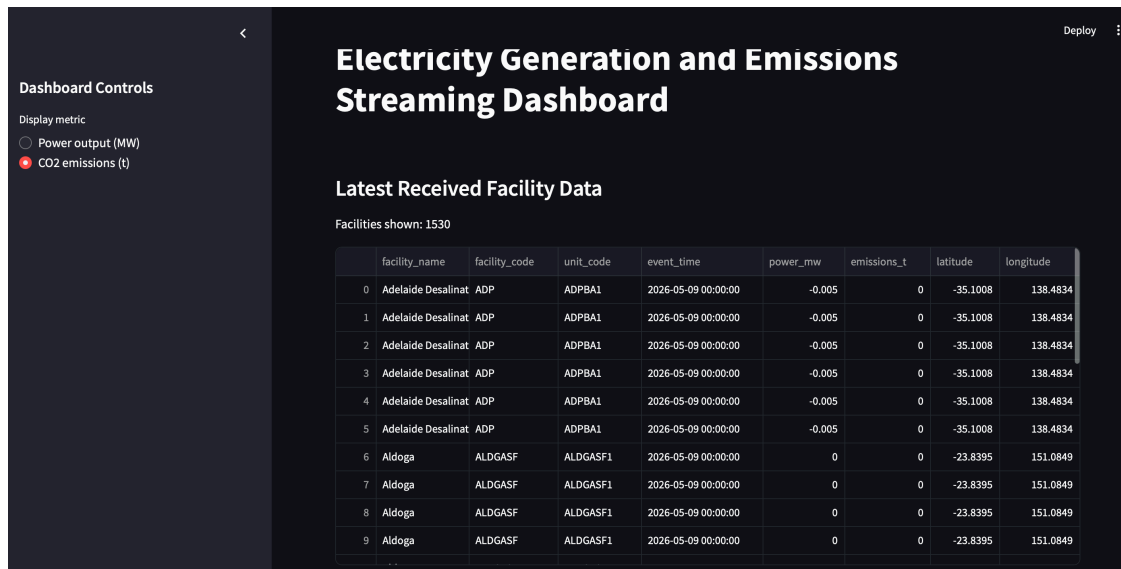
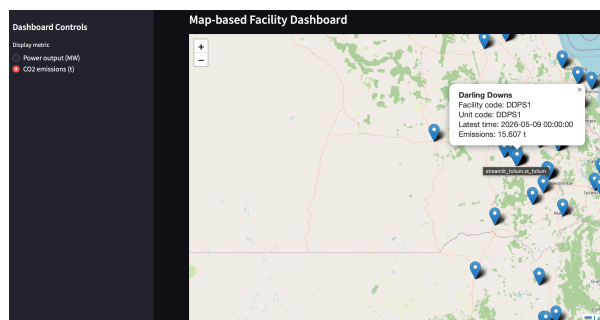


Figure 2: Main Streamlit dashboard showing live subscribed facility data

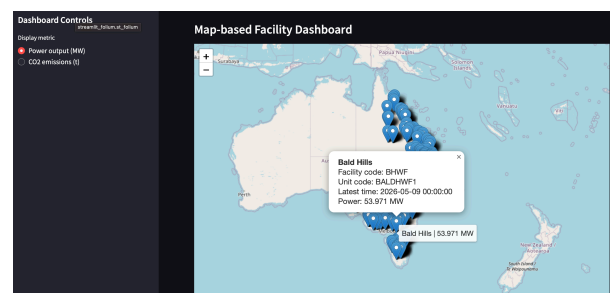
The dashboard successfully displayed facility name, facility code, unit code, timestamp, power generation values, emissions values, and geographical coordinates. This confirmed that MQTT payloads were correctly decoded and integrated into the dashboard system.

An interactive geospatial dashboard was implemented using Folium. Each facility was represented as a clickable map marker based on its latitude and longitude coordinates. Clicking on a marker displayed detailed popup information including facility identity, unit code, latest timestamp, and either power output or emissions values depending on the selected display mode.

Figure 3 shows the dashboard operating under emissions mode and power output mode.



(a) Emissions mode



(b) Power output mode

Figure 3: Interactive dashboard visualisation under different metric modes

The successful display of dynamic popup information confirmed that MQTT messages were subscribed correctly, JSON payloads were decoded successfully, and facility-level integration between Assignment 1 and the streaming system functioned correctly. **The success-**

ful rendering of dynamic facility popups confirmed that MQTT messages were correctly subscribed, decoded, and integrated with the Assignment 1 facility schema.

5 Challenges and Future Improvements

Several challenges were encountered during development. One challenge involved handling facilities with multiple generating units. This was resolved by preserving the original `unit_code` throughout the integration and MQTT publishing stages instead of aggregating facilities prematurely.

Another challenge involved maintaining stable continuous streaming behaviour while avoiding excessive API requests. To address this issue, a cached integrated dataset was materialised locally and replayed repeatedly through MQTT rather than requesting fresh API data during every streaming cycle.

Synchronising dashboard updates with MQTT replay timing also required debugging during development. Testing with separate publisher and subscriber terminals helped verify that records were streamed, subscribed, and decoded correctly before integration into the Streamlit dashboard.

Future improvements could include integrating live API refresh during replay cycles, supporting larger-scale distributed streaming platforms such as Kafka, and improving dashboard analytics with historical trend visualisation and filtering capabilities.

Individual Contribution

Wei Gong: Complete Tasks 1 and 2 and write the corresponding reports. Also, create a GitHub project for group collaboration.

Raina Hong: Complete Tasks 3 and 6 and write the corresponding reports. Also, create a LaTeX project for group report.

Zongquan Tang: Complete Tasks 4 and 5 and write the corresponding reports. Also, write System Overview, Challenges and Future Improvements and revise the draft report into a final 5-page version.

Acknowledgment of AI Use

In this assignment, we used ChatGPT as a tool to help us organize and format the report, as well as to remove unnecessary statements.

A Appendix: Additional Development and Debugging Screenshots

The following figures provide supplementary evidence of MQTT communication, dashboard deployment, and broker configuration during development.

```
Comp5339_A2 — streamlit run dashboard_app.py — 80x24
Received: Goyder South 65.05
Received: Gullen Range 0.0
Received: Gunning 4.9
Received: Gunsynd 0.0
Received: Hallett 1 18.3
Received: Hallett 2 11.92
Received: Hamilton 0.0
Received: Haughton 0.0
Received: Hayman 0.0
Received: Hazelwood BESS -0.4
Received: Hawkesdale 25.21
Received: Hunter Economic Zone 0.0
Received: Hillston Sun 0.0
Received: Hornsdale 0.0
Received: Hornsdale 2 0.0
Received: Hornsdale 3 0.0
Received: Hornsdale Power Reserve 0.0
Received: Hughenden -0.1
Received: Hume 5.0281
Received: Hume 0.0
Received: Hunter 0.0
Received: Happy Valley -0.005
Received: John Butters 0.0
Received: Jeeralang A 0.0
```

(a) MQTT subscriber verification

```
Comp5339_A2 — mosquitto -v — 80x24
Last login: Sun May 24 14:28:06 on ttys005
(base) tangzongquan@tangzongquandeMacBook-Air ~ % cd /Users/tangzongquan/Desktop /Comp5339_A2
(base) tangzongquan@tangzongquandeMacBook-Air Comp5339_A2 % mosquitto -v
1779598104: Info: running mosquitto as user: tangzongquan.
1779598104: mosquitto version 2.1.2 starting
1779598104: Using default config.
1779598104: Bridge support available.
1779598104: Persistence support available.
1779598104: TLS support available.
1779598104: TLS-PSK support available.
1779598104: Websockets support available.
1779598104: Starting in local only mode. Connections will only be possible from clients running on this machine.
1779598104: Create a configuration file which defines a listener to allow remote access.
1779598104: For more details see https://mosquitto.org/documentation/authentication-methods/
1779598104: Opening ipv4 listen socket on port 1883.
1779598104: Opening ipv6 listen socket on port 1883.
1779598104: mosquitto version 2.1.2 running
1779598126: New connection from ::1:55897 on port 1883.
1779598126: New client connected from ::1:55897 as comp5339_mqtt_publisher (p4,
```

(b) Mosquitto broker configuration

Figure 4: Additional MQTT development and debugging screenshots

```
Comp5339_A2 — streamlit run dashboard_app.py — 80x24

Email:

You can find our privacy policy at https://streamlit.io/privacy-policy

Summary:
- This open source library collects usage statistics.
- We cannot see and do not store information contained inside Streamlit apps, such as text, charts, images, etc.
- Telemetry data is stored in servers in the United States.
- If you'd like to opt out, add the following to ~/.streamlit/config.toml, creating that file if necessary:

[browser]
gatherUsageStats = false

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.75.134:8501
```

Figure 5: Initial Streamlit dashboard deployment and local URL generation